# APPRENDRE ALEX DE ZÉRO

Alex Lang



V1.3

Alexandre Koffi

# APPRENDRE ALEX DE ZÉRO

*Parcours pratique pour débutants — basé sur le projet ALEX*

Objectif : apprendre progressivement à écrire, lancer, tester et organiser des programmes en ALEX. Les exemples reprennent la syntaxe visible dans les exemples et la documentation du projet fourni. Les commandes exactes peuvent varier selon le répertoire et la version installée.

## Comment utiliser ce guide

- Suivez les leçons dans l'ordre et tapez vous-même chaque exemple.
- Enregistrez les fichiers avec l'extension .al, par exemple bonjour.al.
- Depuis la racine du projet ALEX, utilisez la commande indiquée par votre installation : ./alexc run fichier.al ou alexc run fichier.al.
- Si une commande échoue, vérifiez d'abord le dossier courant, le nom du fichier, puis la disponibilité du module utilisé.
- Ne supposez pas qu'une fonction ou un module est disponible dans toutes les versions : vérifiez la documentation et les exemples livrés avec votre version.



# Leçon 1 — Votre premier programme

Un programme est une suite d'instructions. Dans ALEX, print(...) permet d'afficher un résultat.

```
print("Bonjour, monde !")
print("Je découvre le langage ALEX.")
```

Enregistrez ce code dans bonjour.al, puis lancez-le depuis la racine du projet :

```
./alexc run bonjour.al
# ou, selon votre installation :
alexc run bonjour.al
```

Résultat attendu : les deux phrases s'affichent dans le terminal.

**À pratiquer :** Remplacez le texte par votre prénom, puis affichez trois lignes différentes.

# Leçon 2 — Variables et valeurs

Une variable garde une valeur que le programme peut réutiliser. Les exemples du projet utilisent notamment := pour initialiser une variable et = pour modifier une variable déjà créée.

```
nom := "Awa"
age := 18
ville := "Abidjan"
print(nom)
print(age)
print(ville)
age = age + 1
print(age)
```

Les valeurs peuvent être des nombres, du texte, des booléens (true/false), ou des collections comme des listes.

```
actif := true
score := 12.5
nombres := [2, 4, 6, 8]
print(actif, score, nombres)
```

**À pratiquer :** Créez les variables produit, prix et stock, puis affichez-les. Augmentez ensuite le stock de 5.

# Leçon 3 — Calculs et comparaisons

```
a := 10
b := 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a > b)
print(a == b)
```

Les opérateurs courants sont +, -, *, / et % (reste de division). Les comparaisons comme >, <, == et != produisent un résultat vrai ou faux.

**À pratiquer :** Calculez le prix total de 4 articles à 250 chacun. Affichez aussi le reste de 17 divisé par 5.

# Leçon 4 — Conditions

Une condition permet d'exécuter un bloc seulement si une expression est vraie. ALEX utilise une syntaxe à indentation : les lignes du bloc sont décalées.

```
note := 14
if note >= 10:
    print("Admis")
else:
    print("À reprendre")
```

L'indentation fait partie de la structure : gardez une indentation régulière et évitez de mélanger tabulations et espaces.

**À pratiquer :** Écrivez un programme qui affiche « majeur » si age >= 18, sinon « mineur ».

## Leçon 5 — Boucles et listes

Une boucle for parcourt les éléments d'une liste. L'exemple ci-dessous reprend la forme utilisée dans les exemples ALEX.

```
fruits := ["mangue", "banane", "orange"]
for fruit in fruits:
    print(fruit)
```

Une boucle peut aussi parcourir une plage numérique :

```
for n in 1..5:
    print(n)
```

Pour accumuler un total, initialisez une variable avant la boucle :

```
nombres := [3, 5, 7]
total := 0
for n in nombres:
    total += n
print(total)
```

**À pratiquer :** Parcourez une liste de cinq prix et calculez leur somme.

## Leçon 6 — Fonctions

Une fonction regroupe des instructions réutilisables. func introduit une fonction et return renvoie un résultat.

```
func saluer(nom):
    print("Bonjour " + nom)

func carre(n):
    return n * n

saluer("Mariam")
print(carre(6))
```

Les paramètres sont les informations reçues par la fonction. Une fonction peut afficher un résultat, renvoyer une valeur, ou faire les deux.

**À pratiquer :** Créez une fonction double(n) qui renvoie n * 2, puis appelez-la avec plusieurs nombres.

# Leçon 7 — Chaînes de caractères et affichage et f-string

```
prenom := "Koffi"
age := 20
print("Prénom :", prenom)
print(f"{prenom} a {age} ans.")
```

La forme f"..." permet d'insérer des expressions entre accolades, comme le montrent les exemples du projet. Pour joindre du texte, l'opérateur + est également utilisé.

**À pratiquer :** Affichez une phrase qui présente un produit, son prix et sa quantité.

# Leçon 8 — Utiliser un module

Les modules regroupent des fonctionnalités. Le projet fourni contient des exemples qui chargent des modules avec use, par exemple use math ou use json.

```
use math
print(math.abs(-7))
print(math.max(4, 9))
```

Exemple JSON:

```
use json
texte := "{\"projet\": \"Alex\", \"version\": 1.5}"
objet := json.parse(texte)
print(objet["projet"])
print(json.stringify(objet))
```

Si le module ne se charge pas, vérifiez qu'il est présent dans la bibliothèque standard de votre version et que l'environnement de lancement connaît son chemin.

**À pratiquer :** Essayez les fonctions mathématiques.

## Leçon 9 — Gérer une erreur

Un programme robuste doit prévoir les erreurs. Les exemples du projet utilisent try/catch et raise.

```
try:
    valeur := 10 / 0
    print(valeur)
catch e:
    print("Une erreur a été rencontrée :")
    print(e)
```

Le comportement exact des erreurs dépend de l'opération et de la version du runtime. Testez le cas réel dans votre installation et ne masquez pas silencieusement les erreurs.

**À pratiquer :** Entourez une opération potentiellement invalide avec try/catch et affichez un message explicite.

## Leçon 10 — Classes et objets

ALEX prend en charge les classes dans les exemples fournis. Une classe décrit des données et des méthodes ; un objet est une instance de cette classe.

```
class Compteur:
    valeur := 0

    func init(depart):
        self.valeur = depart

    func ajouter(n):
        self.valeur = self.valeur + n
        return self.valeur

c := Compteur(5)
print(c.ajouter(3))
```

Commencez par maîtriser les variables et fonctions avant les classes.

**À pratiquer :** Créez une classe Produit avec un nom, un prix et une méthode qui affiche ses informations.

## Leçon 11 — Lire et écrire des fichiers : étape suivante

Le dépôt ALEX contient un module de fichiers. Avant d'écrire du code dépendant d'une API précise, ouvrez la documentation de votre version et les exemples du module file. Les noms exacts des fonctions ne doivent pas être devinés.

- Repérez le module et son README ou sa documentation.
- Cherchez un exemple qui lit un fichier, puis un exemple qui écrit un fichier.
- Reproduisez l'exemple minimal avant de l'intégrer à votre programme.
- Prévoyez les erreurs : fichier absent, droits insuffisants ou chemin incorrect.

## Leçon 12 — Tester et déboguer

Le projet documente des commandes de développement comme run, check, test, fmt, doc, repl et debug. Leur disponibilité dépend de l'exécutable et de la version installés.

```
alexc run bonjour.al
alexc check bonjour.al
alexc test
```

- run : exécuter un programme.
- check : demander une vérification statique si la commande est prise en charge.
- test : lancer les tests du projet ou les tests découverts.
- fmt : formater le code si cette commande est disponible.
- debug : lancer les outils de débogage documentés par le SDK.

Pour un nouveau projet, le README du dépôt fourni montre aussi la création par modèle, par exemple : alexc new cli mon_app. Consultez l'aide de la commande avant d'utiliser des options avancées.

# Mini-projet final — Gestionnaire de notes

Construisez un petit programme qui stocke des notes dans une liste, calcule la moyenne et indique si la moyenne atteint 10. Cette activité réutilise variables, listes, boucles, conditions et fonctions.

```
notes := [12, 15, 8, 10]
total := 0
for note in notes:
    total += note
moyenne := total / len(notes)
print(f"Moyenne : {moyenne}")
if moyenne >= 10:
    print("Résultat : admis")
else:
    print("Résultat : à reprendre")
```

Si len n'est pas disponible sans import dans votre configuration, consultez les exemples du dépôt : ils montrent notamment `from core use len`.

**À pratiquer :** Améliorez le projet : affichez la meilleure note, la plus faible, et le nombre de notes.

# SUITE DU PARCOURS — ALEX INTERMÉDIAIRE

Cette suite complète les douze premières leçons. Lancez chaque exemple avec votre version d'alexc et vérifiez les modules disponibles.

## Leçon 13 — Annotations de types

Les annotations permettent de préciser le type attendu pour une variable, un paramètre ou une valeur de retour. Elles sont facultatives dans ALEX, mais alexc check peut vérifier les types et certains noms avant l'exécution.

```
age: int := 19
nom: str := "Awa"
actif: bool := true

func doubler(n: int) -> int:
    return n * 2

print(doubler(age))
```

Exemples de types documentés : int, float, bool, str, list[T] et dict[K, V]. Le type optionnel T? accepte aussi null. Une annotation incorrecte peut produire une erreur de vérification.

```
prix: float := 1250.5
nombres: list[int] := [2, 4, 6]
limite: int? := null
```

**À pratiquer :** Ajoutez des annotations à une fonction qui reçoit deux entiers et renvoie leur somme. Essayez ensuite une valeur incompatible et lancez alexc check fichier.al.

# Leçon 14 — Compréhensions de listes

Une compréhension crée une nouvelle liste à partir d'une collection ou d'un intervalle. Elle peut transformer les valeurs et filtrer celles qui vous intéressent.

```
nombres := [1, 2, 3, 4, 5]
carres := [n * n for n in nombres]
pairs := [n for n in nombres if n % 2 == 0]

print(carres)
print(pairs)
```

Une compréhension peut aussi parcourir un intervalle :

```
doubles := [n * 2 for n in 0..5]
print(doubles)
```

Pour les compréhensions de dictionnaires, la forme documentée est {clé: valeur for ...}.

```
carres_dict := {n: n * n for n in 0..4}
print(carres_dict)
```

Dans cet exemple, la variable nom doit être définie avant utilisation si vous souhaitez l'utiliser comme clé ; adaptez donc l'expression à une liste de noms réelle.

**À pratiquer :** À partir d'une liste de prix, créez une liste qui ne contient que les prix supérieurs à 1 000.

# Leçon 15 — Fonctions anonymes et lambdas

Une lambda est une fonction courte dont le corps est une expression. Elle est utile lorsqu'une petite fonction doit être stockée dans une variable ou transmise à une autre fonction.

```
carre := n => n * n
additionner := (a, b) => a + b
constante := () => 7

print(carre(5))
print(additionner(3, 4))
print(constante())
```

La documentation ALEX décrit aussi les fermetures : une fonction peut conserver l'accès à une variable du contexte où elle a été créée.

```
func creer_compteur():
    valeur := 0
    func incrementer():
        valeur = valeur + 1
        return valeur
    return incrementer

compteur := creer_compteur()
print(compteur())
print(compteur())
```

**À pratiquer :** Créez une lambda qui multiplie un nombre par 10. Testez-la avec trois valeurs.

# Leçon 16 — Manipuler le JSON

JSON sert à représenter des données structurées, par exemple une configuration ou les informations d'un produit. Le module json du projet fournit parse, stringify, pretty, load et save.

```
use json

texte := "{\"nom\": \"Clavier\", \"prix\": 5000}"
produit := json.parse(texte)

print(produit["nom"])
print(produit["prix"])
print(json.pretty(produit))
```

json.parse transforme un texte JSON en valeur ALEX. json.stringify transforme une valeur en texte JSON. json.load et json.save utilisent un chemin de fichier ; vérifiez les permissions et le comportement de votre environnement.

```
use json

configuration := {"application": "ALEX", "version": 1}
json.save("config.json", configuration)
relue := json.load("config.json")
print(relue["application"])
```

**À pratiquer :** Créez un objet JSON contenant le nom d'un élève et sa moyenne, puis affichez ces deux champs.

# Leçon 17 — Utiliser les arguments de la ligne de commande

Le module argparse permet de définir les arguments acceptés par un programme. Les options de add_argument sont transmises dans un dictionnaire, comme dans la documentation du projet.

```
use argparse

p := argparse.parser(prog="salut", description="Exemple ALEX")
p.add_argument("nom", {"help": "nom de la personne"})
args := p.parse_args()
print("Bonjour", args["nom"])
```

Lancez le programme avec un argument, par exemple :

```
alexc run salut.al Awa
```

Les possibilités exactes dépendent de la version installée. La documentation du dépôt décrit aussi les options, les valeurs par défaut, les choix autorisés et les messages d'aide.

**À pratiquer :** Ajoutez un argument numérique avec le type int et affichez le double de cette valeur.

# Leçon 18 — Premiers pas avec l’asynchrone

Les fonctions async func produisent des tâches. await attend leur résultat. Dans l’implémentation documentée, les coroutines coopèrent sur un seul thread et cèdent la main lorsqu’elles attendent un futur non terminé.

```
use asyncio

async func calculer(n):
    await asyncio.sleep(0.1)
    return n * 2

async func main():
    a := calculer(10)
    b := calculer(21)
    print(await a, await b)

await main()
```

N’utilisez async/await que si le module asyncio et cette syntaxe sont disponibles dans votre version. Une fonction asynchrone doit être attendue pour récupérer son résultat ; appeler la fonction ne revient pas à obtenir directement le résultat final.

**À pratiquer :** Modifiez l’exemple pour calculer deux autres valeurs et afficher les résultats.

# Leçon 19 — Premiers pas avec le réseau HTTP

Le projet contient un paquet network organisé en modules. La documentation donne notamment un client HTTP et un serveur HTTP. Une requête réseau peut échouer : prévoyez des erreurs et des délais d'attente.

```
use network.http_client as hc

client := hc.Client("https://api.exemple.org/", {}, 10.0)
reponse := client.get("")
print(reponse)
```

Cet exemple illustre la forme générale de l'API documentée ; l'hôte d'exemple n'est pas une API de démonstration garantie. Pour un vrai test, choisissez un service que vous êtes autorisé à contacter et consultez la documentation du module pour les méthodes de réponse disponibles.

La documentation présente également un serveur local sous la forme hs.App(), app.get(...) et hs.Server(...). Ne publiez pas un serveur sur Internet sans traiter l'authentification, la validation des entrées, TLS et les erreurs.

**À pratiquer :** Lisez NETWORK.md et identifiez les modules dédiés à HTTP, TLS, DNS et aux sockets.

# Leçon 20 — Vérifier et organiser un projet

Avant d'ajouter une fonctionnalité, vérifiez que le programme est valide, puis exécutez-le et testez les cas importants. La documentation du projet mentionne les commandes run, check et test, selon la version d'alexc.

```
alexc check mon_programme.al
alexc run mon_programme.al
alexc test
```

Une organisation simple pour un petit projet peut séparer le point d'entrée, les fonctions métier et les données. Vérifiez dans votre installation la syntaxe exacte des imports et la manière dont le chargeur trouve les modules.

```
mon_projet/
  main.al
  calculs.al
  README.md
```

Gardez des noms explicites, ajoutez des commentaires lorsqu'ils apportent une information utile et testez les cas limites : liste vide, valeur nulle, texte inattendu ou fichier absent.

**À pratiquer :** Créez un programme de calcul dans main.al, déplacez une fonction réutilisable dans un module séparé selon la syntaxe prise en charge, puis vérifiez les deux fichiers.

## Mini-projet 2 — Carnet de produits

Objectif : réutiliser les variables, les listes, les boucles, les fonctions et JSON pour afficher un petit catalogue. Cette version pédagogique garde les données en mémoire ; elle ne remplace pas une base de données.

```
print(produit["nom"], ":", produit["prix"], "FCFA")
```

À améliorer : demandez une quantité par produit, calculez le montant total, ajoutez une recherche par nom et enregistrez les données au format JSON après avoir vérifié les fonctions de fichier disponibles.

## Méthode de progression sur 4 semaines

- Semaine 1 : leçons 1 à 4. Écrivez de petits programmes chaque jour.
- Semaine 2 : leçons 5 à 7. Entraînez-vous avec boucles, listes, fonctions et chaînes.
- Semaine 3 : leçons 8 à 10. Explorez les modules, les erreurs et les objets.
- Semaine 4 : leçons 11 à 12 et mini-projet. Lisez les exemples existants et écrivez vos propres tests.

## Plan de progression — semaines 5 à 8

- Semaine 5 : leçons 13 et 14 — annotations de types et compréhensions.
- Semaine 6 : leçons 15 et 16 — lambdas, fermetures et JSON.
- Semaine 7 : leçons 17 et 18 — arguments de commande et asynchrone.
- Semaine 8 : leçons 19 et 20 — réseau, vérification et mini-projet.

Note importante : ce guide est un parcours pédagogique initial. Vérifiez les commandes avec la version exacte d'ALEX installée et signalez les erreurs obtenues pour adapter les leçons.